

Package ‘ComparatorSelectionExplorer’

September 26, 2025

Type Package

Title OHDSI Comparator Selection Explorer

Version 1.0.0.99999

Date 2022-11-29

Maintainer Jamie Gilbert <gilbert@ohdsi.org>

Description Utility to select comparators for new user cohort population effect estimation.

License Apache License

VignetteBuilder knitr

Encoding UTF-8

Depends R (>= 4.1.0)

Imports DatabaseConnector (>= 6.0.0),
ParallelLogger,
ResultModelManager (>= 0.4.0),
checkmate,
SqlRender (>= 1.9.0),
readr,
dplyr,
digest,
purrr,
CohortGenerator,
shiny,
shinyjs,
reactable,

Suggests scales,
ggplot2,
plotly,
htmltools,
shinycssloaders,
shinydashboard,
stringr,
tippy,
tidyr,
Eunomia,
testthat (>= 3.0.0),
knitr,
withr,
reticulate,
duckdb

Remotes OHDSI/CohortGenerator@develop

BugReports <https://github.com/OHDSI/ComparatorSelectionExplorer/issues>

Copyright See file COPYRIGHTS

Config/testthat/edition 3

RoxygenNote 7.3.3

Contents

createExecutionSettings	2
createResultsDataModel	4
createResultsQueryNamespace	5
createShinyApp	6
execute	6
exportResults	7
generateSimilarityScores	7
getCohortDefinitions	8
getCohortDefinitionsTable	8
getCohortSimilarityScores	9
getCoOccurenceTableData	9
getDatabaseSimilarityScores	10
getDatabaseSources	11
getDataMigrator	11
getDbCosineSimilarityTable	12
getDbDataSourcesTable	13
getPairwiseCovariateData	13
getResultsDataModelSpec	14
launchShinyApp	14
migrateDataModel	15
uploadResults	15
Index	16

createExecutionSettings
<i>createExecutionSettings</i>

Description

executions settings to be used throughout package for analysis

Usage

```
createExecutionSettings(
  connectionDetails = NULL,
  connection = NULL,
  databaseName = NULL,
  databaseId = NULL,
  cdmDatabaseSchema,
  vocabularyDatabaseSchema = cdmDatabaseSchema,
```

```

resultsDatabaseSchema,
cohortDatabaseSchema = resultsDatabaseSchema,
cohortTable,
tempEmulationSchema = getOption("sqlRenderTempEmulationSchema"),
cohortDefinitionSet = NULL,
targetCohortIds = NULL,
cohortCountTable = "cse_cohort_count",
cohortDefinitionTable = "cse_cohort_definition",
covariateDefTable = "cse_covariate_ref",
covariateMeansTable = "cse_covariate_means",
cosineSimStratifiedTable = "cse_cosine_sim",
minExposureSize = 1000,
logFileLocation = paste0("cse-execution-log-", cdmDatabaseSchema, ".txt"),
exportDir = tempfile(),
removeExportDir = TRUE,
exportZipFile = file.path(normalizePath(getwd()), paste0("cse_results_",
  cdmDatabaseSchema, ".zip"))
)

```

Arguments

connectionDetails	DatabaseConnector::connectionDetails instance
connection	DatabaseConnector::connection (defaults to null). Use for persistent storage of reference
databaseName	Database identifier (string)
databaseId	Database identifier integer (optional)
cdmDatabaseSchema	String Database schema - where data lives
vocabularyDatabaseSchema	standard vocabulary database schema
resultsDatabaseSchema	String Database schema - where similarity scores are stored
cohortDatabaseSchema	String Database schema - where cohort table is created
cohortTable	cohort table for exposures
tempEmulationSchema	String DatabaseSchema - temp emulation schema for oracle, bigquery
cohortDefinitionSet	CohortGenerator::cohortDefinitionSet - intended to be custom exposures or indication cohorts
targetCohortIds	(optional) Integer ids for cohorts to limit cosine similarity calculation to Must be a valid RxNorm ingredient, ATC class or included in the cohortDefinitionSet
cohortCountTable	(optional) count tabs
cohortDefinitionTable	(optional) definitions table
covariateDefTable	(optional) where covariate definitions are stored

`createResultsQueryNamespace`*Create Query Namespace for Results Data Model*

Description

This function initializes a query namespace object for use with the Results data model. It loads the data model specification from a CSV file and calls `ResultModelManager::createQueryNamespace()` with the provided connection and schema info.

Usage

```
createResultsQueryNamespace(  
  connectionDetails,  
  resultsSchema,  
  tablePrefix = "",  
  dataModelSpecPath = system.file("settings", "resultsDataModel.csv", package =  
    "ComparatorSelectionExplorer"),  
  usePooledConnection = FALSE  
)
```

Arguments

<code>connectionDetails</code>	A <code>DatabaseConnector</code> connection details object.
<code>resultsSchema</code>	The database schema where the results tables are located.
<code>tablePrefix</code>	Optional table prefix for the results tables (default is "").
<code>dataModelSpecPath</code>	Path to the results data model specification CSV file (default is "resultsDataModel.csv").
<code>usePooledConnection</code>	Logical; whether to use pooled connections (default is FALSE).

Value

A query namespace object as returned by `ResultModelManager::createQueryNamespace()`.

Examples

```
## Not run:  
connectionDetails <- DatabaseConnector::createConnectionDetails(dbms = "postgresql", ...)  
qns <- createResultsQueryNamespace(  
  connectionDetails = connectionDetails,  
  resultsSchema = "my_results_schema"  
)  
  
## End(Not run)
```

createShinyApp	<i>Launch the Comparator Selection Shiny App</i>
----------------	--

Description

Launches the full Shiny application for comparator selection, using the modular UI and server functions.

Usage

```
createShinyApp(connectionDetails, resultsSchema, tablePrefix, ...)
```

Arguments

connectionDetails	DatabaseConnector connection details object.
resultsSchema	Character. Results database schema.
tablePrefix	Character. Optional table prefix for results tables. Default is "".
...	additional parameters to pass to shiny::shinyApp

Value

A Shiny app object (invisibly; called for its side effect of launching the app).

Examples

```
## Not run:
createShinyApp(connectionDetails, "results_schema", tablePrefix = "myPrefix_")

## End(Not run)
```

execute	<i>Execute Package</i>
---------	------------------------

Description

Execute data generation step on a given OMOP CDM - connect, create cohorts and generate data
Data can be transferred with the OhdsiSharing package

Usage

```
execute(executionSettings = NULL, ...)
```

Arguments

executionSettings	settings greated with @seealso createExecutionSettings
...	settings greated with @seealso createExecutionSettings

exportResults	<i>Export Results</i>
---------------	-----------------------

Description

Create a zip file containing csvs of computed results tables

Usage

```
exportResults(executionSettings = NULL, ...)
```

Arguments

executionSettings	settings greated with @seealso createExecutionSettings
...	settings greated with @seealso createExecutionSettings

generateSimilarityScores	<i>Generate similarity scores</i>
--------------------------	-----------------------------------

Description

create cosine similarity scores

Usage

```
generateSimilarityScores(executionSettings = NULL, ...)
```

Arguments

executionSettings	settings greated with @seealso createExecutionSettings
...	settings greated with @seealso createExecutionSettings

getCohortDefinitions *Get Cohort Definitions*

Description

Queries the database for available cohort definitions.

Usage

```
getCohortDefinitions(qns)
```

Arguments

qns A QueryNamespace object.

Value

A data.frame of cohort definitions.

Examples

```
# qns <- createResultsQueryNamespace(...)
# getCohortDefinitions(qns)
```

getCohortDefinitionsTable
 Get Cohort Definitions Table for a Database

Description

Returns a table of cohort definitions and counts for a specified database.

Usage

```
getCohortDefinitionsTable(qns, databaseId, counts = TRUE)
```

Arguments

qns A QueryNamespace object.
databaseId Character or integer. The database ID(s) to fetch data for.
counts Logical. Whether to include counts (default TRUE).

Value

A data.frame of cohort definitions and counts.

Examples

```
# getCohortDefinitionsTable(qns, databaseId = "CCAE")
```

```
getCohortSimilarityScores
```

Get Cohort Similarity Scores

Description

Retrieves similarity scores for all comparators to a target cohort.

Usage

```
getCohortSimilarityScores(qns, targetCohortId, weights = NULL)
```

Arguments

qns	A QueryNamespace object.
targetCohortId	Integer or character. The target cohort definition ID.
weights	optional named vector of numeric weights Demographics, "Medical history" "Presentation" "prior meds" "visit context" with numeric values to adjust

Value

A data.frame with similarity scores.

Examples

```
# getCohortSimilarityScores(qns, 101)
```

```
getCoOccurrenceTableData
```

Get Co-occurrence Table Data

Description

Retrieves covariate data for covariates that occur on the same day as the exposure event.

Usage

```
getCoOccurrenceTableData(
  qns,
  databaseIds,
  prevInputHighMax,
  prevInputHighMin,
  prevInputLowMax,
  prevInputLowMin,
  cohortDefinitionId1,
  cohortDefinitionId2
)
```

Arguments

qns A QueryNamespace object.
 databaseIds Character vector of database IDs.
 prevInputHighMax Numeric. High maximum prevalence threshold.
 prevInputHighMin Numeric. High minimum prevalence threshold.
 prevInputLowMax Numeric. Low maximum prevalence threshold.
 prevInputLowMin Numeric. Low minimum prevalence threshold.
 cohortDefinitionId1 Target cohort definition ID.
 cohortDefinitionId2 Comparator cohort definition ID.

Value

A data.frame with co-occurrence covariate data.

Examples

```
# getCoOccurrenceTableData(qns, ...)
```

```
getDatabaseSimilarityScores
```

Get Database Similarity Scores

Description

Retrieves similarity scores for a target cohort across specific databases.

Usage

```
getDatabaseSimilarityScores(qns, targetCohortId, databaseIds)
```

Arguments

qns A QueryNamespace object.
 targetCohortId Integer or character. The target cohort definition ID.
 databaseIds Character vector of database IDs.

Value

A data.frame with similarity scores for the specified databases.

Examples

```
# getDatabaseSimilarityScores(qns, 101, c("CCAE", "MDCR"))
```

getDatabaseSources	<i>Get All Database Sources</i>
--------------------	---------------------------------

Description

Retrieves all database source metadata from the results schema.

Usage

```
getDatabaseSources(qns)
```

Arguments

qns	A QueryNamespace object.
-----	--------------------------

Value

A data.frame of all database sources.

Examples

```
# getDatabaseSources(qns)
```

getDataMigrator	<i>Get database migrations instance</i>
-----------------	---

Description

Returns ResultModelManager DataMigrationsManager instance.

Usage

```
getDataMigrator(connectionDetails, databaseSchema, tablePrefix = "")
```

Arguments

connectionDetails	DatabaseConnector connection details object
databaseSchema	String schema where database schema lives
tablePrefix	(Optional) Use if a table prefix is used before table names (e.g. "cd_")

Value

Instance of ResultModelManager::DataMigrationManager that has interface for converting existing data models

`getDbCosineSimilarityTable`*Get Cosine Similarity Table for a Database*

Description

Retrieves cosine similarity values for a target and comparator cohort in a specific database.

Usage

```
getDbCosineSimilarityTable(  
  qns,  
  targetCohortId,  
  comparatorCohortId,  
  databaseId,  
  weights = NULL,  
  returnReactable = FALSE,  
  fmtSim = "%.3f"  
)
```

Arguments

<code>qns</code>	A QueryNamespace object.
<code>targetCohortId</code>	Target cohort definition ID.
<code>comparatorCohortId</code>	Comparator cohort definition ID.
<code>databaseId</code>	Character or integer. The database ID.
<code>weights</code>	a set of numeric weights > 0 to scale each covariate domain in the cosine similarity calculation (optional)
<code>returnReactable</code>	Logical. If TRUE, returns a Reactable table; otherwise returns a data.frame.
<code>fmtSim</code>	number of significant figures to format float output

Value

A Reactable table or data.frame of cosine similarity values.

Examples

```
# getDbCosineSimilarityTable(qns, 101, 202, "CCAE", TRUE)
```

getDbDataSourcesTable *Get Data Source Table for All Databases*

Description

Retrieves metadata about all database sources, optionally as a Reactable table.

Usage

```
getDbDataSourcesTable(qns, reactableTable = TRUE)
```

Arguments

qns A QueryNamespace object.

reactableTable Logical. If TRUE (default), returns a Reactable table; otherwise returns a data.frame.

Value

A Reactable table or data.frame of database sources.

Examples

```
# getDbDataSourcesTable(qns)
```

getPairwiseCovariateData
Get Pairwise Covariate Data

Description

Retrieves covariate mean and standardized difference for a pair of cohorts in a database.

Usage

```
getPairwiseCovariateData(
  qns,
  databaseId,
  cohortDefinitionId1,
  cohortDefinitionId2
)
```

Arguments

qns A QueryNamespace object.

databaseId Character or integer. The database ID.

cohortDefinitionId1 Target cohort definition ID.

cohortDefinitionId2 Comparator cohort definition ID.

Value

A data.frame with covariate means and std. differences.

Examples

```
# getPairwiseCovariateData(qns, "CCA", 101, 202)
```

```
getResultsDataModelSpec
```

Get Results Data Model Specifications

Description

Get Results Data Model Specifications

Usage

```
getResultsDataModelSpec()
```

```
launchShinyApp
```

Launch the Comparator Selection Shiny App

Description

Launches the full Shiny application for comparator selection, using the modular UI and server functions.

Usage

```
launchShinyApp(connectionDetails, resultsSchema, tablePrefix, ...)
```

Arguments

connectionDetails

DatabaseConnector connection details object.

resultsSchema Character. Results database schema.

tablePrefix Character. Optional table prefix for results tables. Default is "".

... additional parameters to pass to shiny::shinyApp

Value

A Shiny app object (invisibly; called for its side effect of launching the app).

Examples

```
## Not run:
launchShinyApp(connectionDetails, "results_schema", tablePrefix = "myPrefix_")

## End(Not run)
```

migrateDataModel	<i>Migrate Data model</i>
------------------	---------------------------

Description

Migrate data from current state to next state

It is strongly advised that you have a backup of all data (either sqlite files, a backup database (in the case you are using a postgres backend) or have kept the csv/zip files from your data generation.

Usage

```
migrateDataModel(connectionDetails, databaseSchema, tablePrefix = "")
```

Arguments

connectionDetails

DatabaseConnector connection details object

databaseSchema String schema where database schema lives

tablePrefix (Optional) Use if a table prefix is used before table names (e.g. "cd_")

uploadResults	<i>Upload Results</i>
---------------	-----------------------

Description

Upload results to database server

Usage

```
uploadResults(
  connectionDetails,
  databaseSchema,
  zipFileName,
  tablePrefix = "",
  importFilpath = tempfile(),
  ...
)
```

Arguments

connectionDetails

DatabaseConnector connection details object

databaseSchema String schema where database schema lives

zipFileName Path to zipFile containing results

tablePrefix (Optional) Use if a table prefix is used before table names (e.g. "cd_")

importFilpath file path to export zipped results to before upload (optional - default is temporary)

... Elipsis - see ResultModelManager::uploadResults

Index

`createExecutionSettings`, [2](#)
`createResultsDataModel`, [4](#)
`createResultsQueryNamespace`, [5](#)
`createShinyApp`, [6](#)

`execute`, [6](#)
`exportResults`, [7](#)

`generateSimilarityScores`, [7](#)
`getCohortDefinitions`, [8](#)
`getCohortDefinitionsTable`, [8](#)
`getCohortSimilarityScores`, [9](#)
`getCoOccurrenceTableData`, [9](#)
`getDatabaseSimilarityScores`, [10](#)
`getDatabaseSources`, [11](#)
`getDataMigrator`, [11](#)
`getDbCosineSimilarityTable`, [12](#)
`getDbDataSourcesTable`, [13](#)
`getPairwiseCovariateData`, [13](#)
`getResultsDataModelSpec`, [14](#)

`launchShinyApp`, [14](#)

`migrateDataModel`, [15](#)

`uploadResults`, [15](#)